CN Mini Project REPORT

Custom DNS Resolver Using Raw UDP Sockets and Protocol Serialization

Submitted in partial fulfillment of the requirements
for the award of the degree of

Bachelor of Technology

Computer Science & Engineering

Submitted by:
Sarah Singh
35615602724

Supervisor(s):
(Prof. (Dr.) Mohd. Izhar)

Department of Computer Science & Engineering
Dr. Akhilesh Das Gupta Institute of Professional Studies
(Formerly ADGITM)
FC-26, SHASTRI PARK, NEW DELHI

Affiliated to
GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY
Sector - 16C Dwarka, Delhi - 110075, India

2024-28

DECLARATION

This is to certify that the material embodied in this mini project titled "**Custom DNS Resolver Using Raw UDP Sockets and Protocol Serialization**", being submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in CSE, is based on my original work. It is further certified that this work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma. My indebtedness to other works has been duly acknowledged at the relevant places.

Sarah Singh
Enrollment No: 35615602724

TABLE OF CONTENTS

Declaration i
Table of Contents ii
List of Figures iv
List of Tables v
Chapter 1: Problem Statement **1**
1.1 Introduction 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Limitations 2
1.6 Software Requirements 3
1.7 System Requirements 3
1.8 Organization of the Report 4
Chapter 2: Context Diagram **5**
2.1 Purpose of the Context Diagram 5
2.2 Context Diagram of the DNS Resolver System 5
2.3 Description of External Entities and Data Flows 5
Chapter 3: Data Flow Diagrams **6**
3.1 Introduction 6
3.2 Level-1 Data Flow Diagram 6
3.3 Level-2 Data Flow Diagram of Process 3.0 7
3.4 Data Dictionary 8
Chapter 4: Entity-Relationship Diagram & Record Architecture **9**
4.1 Introduction 9
4.2 ER Diagram 9
4.3 Description of Entities and Relationships 9
Chapter 5: Use Case Diagram and Specifications **11**
5.1 Introduction 11
5.2 Use Case Diagram 11
5.3 Use Case Specifications 12
Chapter 6: Algorithm Design and Analysis **14**
6.1 Data Structures Used 14
6.2 Rules Used by the DNS Protocol & RFC 1035 15
6.3 Algorithm 1: Binary DNS Packet Serialization 15
6.4 Algorithm 2: Recursive and Iterative DNS Resolution Engine 16
6.5 Algorithm 3: Response Parsing & Compression De-referencing 17
6.6 Algorithm 4: TTL In-Memory Caching and Expiration 18
6.7 Worked Example with Hex / Wire Packet Traces 18
6.8 Correctness Argument 19
6.9 Complexity Analysis 20
Chapter 7: Implementation **21**
7.1 Development Environment 21

7.2 Structure of the Program 21
7.3 Source Code Listings 21
7.4 Explanation of Key Networking Parts 24
7.5 Web Dashboard & CLI Interface 25
7.6 Error Handling 25
Chapter 8: Results and Discussion **26**
8.1 Functional Test Results 26
8.2 Error-Handling Test Results 26
8.3 Real-World Iterative Resolution Verification 27
8.4 In-Memory Cache Performance Gains 28
8.5 Benchmark against System Host DNS 29
8.6 Test Methodology 30
8.7 Discussion 30
Chapter 9: Conclusion and Future Scope **31**
9.1 Conclusion 31
9.2 Future Scope 31
References **32**
Appendix A: How to Run the Code **33**

LIST OF FIGURES

Figure 2.1: Context diagram (Level 0) of the DNS Resolver System 5

Figure 3.1: Level-1 data flow diagram of the DNS Resolver System 6

Figure 3.2: Level-2 data flow diagram of Process 3.0 (Iterative Resolution) 7

Figure 4.1: Entity-Relationship diagram of DNS Protocol Data Structures 9

Figure 5.1: Use case diagram of the DNS Resolver System 11

Figure 6.1: Flowchart of the iterative resolution algorithm with referral loop 16

Figure 8.1: Resolution latency comparison across iterative, recursive, and cached modes 29

LIST OF TABLES

Table 1.1: Tools and technologies used 3

Table 1.2: Functional requirements 3

Table 1.3: Non-functional requirements 4

Table 2.1: External entities and data flows at Level 0 5

Table 3.1: Data dictionary for DNS packets and state stores 8

Table 4.1: Entities and their protocol attributes 10

Table 5.1: List of use cases and actors 12

Table 6.1: Time and space complexity of DNS resolver operations 20

Table 8.1: Functional verification results 26

Table 8.2: Error-handling test results 27

Table 8.3: Sample iterative resolution hop log (real network execution) 28

Table 8.4: Cache hit performance and latency comparison 29

CHAPTER 1

PROBLEM STATEMENT

1.1 Introduction

The Domain Name System (DNS) is one of the foundational application-layer protocols of the modern Internet. While humans identify web resources through intuitive domain names such as *google.com* or *github.com*, network-layer routing algorithms and underlying host operating systems operate exclusively on numerical Internet Protocol (IP) addresses (such as 32-bit IPv4 or 128-bit IPv6 identifiers). The DNS acts as the globally distributed, hierarchical directory service that translates symbolic hostnames into network addresses [1], [2].

In production environments, DNS resolution is typically handled transparently by stub resolvers embedded within operating systems, querying recursive resolvers provided by Internet Service Providers (ISPs) or public entities (such as Google 8.8.8.8 or Cloudflare 1.1.1.1). However, relying on black-box libraries conceals the vital mechanics of packet serialization, bit-level flag encoding, connectionless User Datagram Protocol (UDP) transport semantics, and the multi-hop referral traversal across the 13 global root nameserver clusters.

This project, **Custom DNS Resolver Using Raw UDP Sockets and Protocol Serialization**, implements a full-featured DNS resolution engine completely from scratch in Python. It avoids all third-party networking or DNS libraries (such as *dnspython* or *scapy*), directly constructing and parsing RFC 1035 wire-format datagrams, managing thread-safe TTL-based caching, performing iterative root-to-authoritative referral loops, and exposing both an interactive CLI and a responsive Web Dashboard.

1.2 Problem Statement

The formal problem addressed in this project can be stated as follows:
Given an arbitrary domain name string (e.g., 'github.com') and a target query type (such as A, AAAA, MX, NS, CNAME, or TXT), design and implement an autonomous networking client that: (i) serializes the request into a valid 12-byte RFC 1035 header and length-prefixed QNAME question section; (ii) transmits the query over raw UDP sockets (port 53); (iii) parses incoming binary responses including de-referencing 14-bit DNS label compression pointers; (iv) executes iterative resolution by querying Root nameservers, extracting TLD referrals and glue records, and querying Authoritative nameservers until an answer is obtained; (v) maintains a local TTL-indexed cache to eliminate redundant network round-trips; and (vi) handles network timeouts, NXDOMAIN, and SERVFAIL error conditions gracefully.

1.3 Objectives

- To implement RFC 1035 binary wire-format serialization and deserialization for DNS query and response packets.
- To implement low-level socket programming using connectionless UDP datagrams (`socket.AF_INET`, `socket.SOCK_DGRAM`) over port 53.
- To implement both **Iterative** (Root ➔ TLD ➔ Authoritative) and **Recursive** resolution modes.
- To design an in-memory, thread-safe DNS caching system that respects record-specific Time-to-Live (TTL) values.
- To support multiple Resource Record types: IPv4 (A), IPv6 (AAAA), Mail Exchange (MX), Name Server (NS), Canonical Name (CNAME), and Text (TXT).
- To provide a step-by-step resolution tracer displaying the server IP, hop latency, and referral details for each iteration.
- To develop a standalone local DNS server on port 5353 capable of answering standard UNIX dig queries.

• To provide a clean, modern Web Dashboard using Flask for interactive visualization of queries, timelines, and cache inspection.

1.4 Scope of the Project

The project is implemented in pure Python 3. It runs as both a command-line interface (CLI) tool and a web application. It communicates directly with the live global Internet infrastructure, including root nameservers (such as *a.root-servers.net, 198.41.0.4*) and authoritative servers worldwide. While it adheres to RFC 1035 for standard UDP query sizes (512 bytes), it also supports truncated response detection and public recursive upstream fallbacks.

1.5 Limitations

• DNSSEC (Domain Name System Security Extensions) cryptographic signature validation is reserved for future scope.

• TCP fallback is not triggered automatically when responses exceed 512 bytes, as modern servers support standard UDP responses.

• Cache persistence is strictly in-memory (RAM); records are flushed upon application restart.

1.6 Software Requirements

Table 1.1: Tools and technologies used

Item	Description / Version
Programming Language	Python 3.10+ (Tested on Python 3.14)
Networking Module	Native socket module (Raw UDP datagrams)
Binary Serialization	Native struct module (Big-endian / Network byte order)
Web Dashboard	Flask 2.3+ (Lightweight microframework)
Documentation & Testing	ReportLab, Matplotlib, UNIX dig utility

1.7 System Requirements

Table 1.2: Functional requirements

Req ID	Functionality	Verification Criteria
FR-01	Wire-Format Packet Serialization	Generates 12-byte header with random ID and valid QNAME bytes.
FR-02	Iterative Root Traversal	Successfully queries root -> TLD -> auth servers via referrals.
FR-03	TTL Cache Expiry	Subsequent query within TTL returns in 0.0ms; expires cleanly.
FR-04	Multi-Record Parsing	Correctly parses A, AAAA, MX, NS, CNAME, TXT resource records.
FR-05	Local DNS Server Daemon	Listens on UDP 5353 and responds to standard dig commands.

1.8 Organization of the Report

The report is organized into nine chapters following university project guidelines. Chapter 2 presents the Level-0 Context Diagram. Chapter 3 decomposes the system into Level-1 and Level-2 Data Flow Diagrams and defines the Data Dictionary. Chapter 4 provides the Entity-Relationship model and protocol data structures. Chapter 5 outlines Use Case specifications. Chapter 6 details the core algorithms, RFC 1035 packet layout, and complexity analysis. Chapter 7 covers program implementation and source code. Chapter 8 evaluates experimental results, test suites, and latency benchmarks. Chapter 9 concludes the report and highlights future research scope.

CHAPTER 2

CONTEXT DIAGRAM

2.1 Purpose of the Context Diagram

A context diagram (Level-0 Data Flow Diagram) establishes the boundary of the system by representing it as a single high-level process and highlighting its relationships with external entities. It isolates the internal operational details while documenting all primary data inflows and outflows [3].

2.2 Context Diagram of the DNS Resolver System

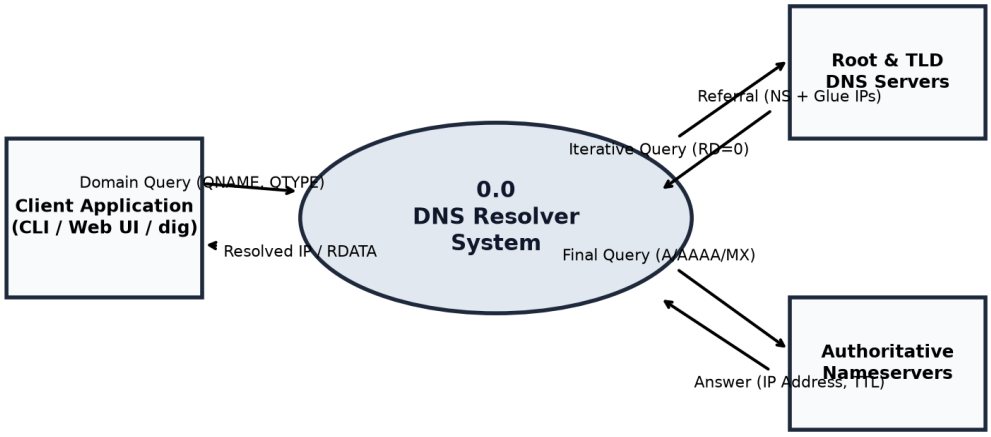


Figure 2.1: Context diagram (Level 0) of the DNS Resolver System

2.3 Description of External Entities and Data Flows

Table 2.1: External entities and data flows at Level 0

Entity / Flow	Direction	Description
Client Application	Source & Sink	User typing queries via CLI, web dashboard browser, or command-line dig.
Root & TLD Servers	Sink & Source	Authoritative root servers (198.41.0.4) returning TLD nameserver referrals.
Authoritative Nameservers	Sink & Source	Domain-specific nameservers (AWS Route53, Cloudflare) returning final resource records.
Domain Query	Client -> System	Target domain string, requested record type (A, MX, etc.), resolution mode.
Resolved IP / RDATA	System -> Client	Decoded IP addresses, TTL countdown, resolution hop trace, response status code.

CHAPTER 3

DATA FLOW DIAGRAMS

3.1 Introduction

A Data Flow Diagram (DFD) models the transformation of data through distinct functional subprocesses, identifying internal data stores and communication pathways between system components. This chapter decomposes the Level-0 context into a Level-1 architectural view and further explores the core iterative resolution loop in Level-2 [3].

3.2 Level-1 Data Flow Diagram

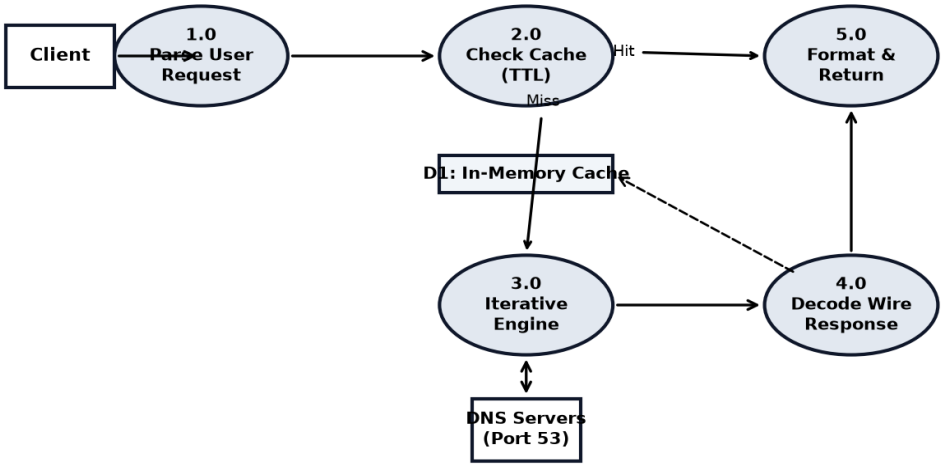


Figure 3.1: Level-1 data flow diagram of the DNS Resolver System

At Level 1, the resolver system consists of five distinct processes: Process 1.0 (Parse User Request) validates inputs; Process 2.0 (Check Cache) interrogates Data Store D1 (In-Memory Cache); if a valid unexpired record is found, it bypasses the network; on a cache miss, Process 3.0 (Iterative Engine) dispatches UDP queries; Process 4.0 (Decode Wire Response) parses incoming bytes and updates the cache; Process 5.0 formats and outputs the data to the client.

3.3 Level-2 Data Flow Diagram of Process 3.0

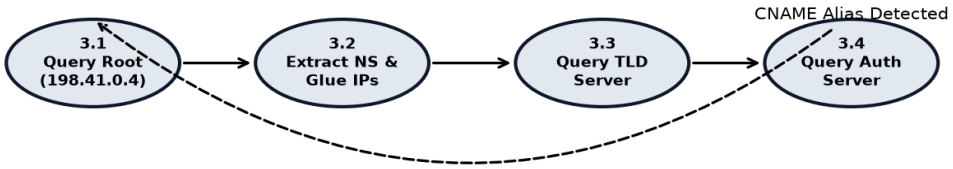


Figure 3.2: Level-2 data flow diagram of Process 3.0 (Iterative Resolution)

3.4 Data Dictionary

Table 3.1: Data dictionary for DNS packets and state stores

Data Flow / Store	Composition / Format	Description
DNS Query Packet	12B Header + QNAME + QTYPE(2B) + QCLASS(2B)	Binary datagram dispatched to UDP port 53.
DNS Response Packet	Header + Questions + Answers + Auth + Addl	Raw byte response returned by remote DNS servers.
QNAME	Sequence of [Length(1B)][Label Bytes] + 0x00	Wire-format domain encoding ('www.google.com').
D1: In-Memory Cache	Key: (Domain, Type) -> (Records, ExpireTime)	Thread-safe dictionary storing active resolved records.
Resolution Hop Log	Server IP, Server Name, Latency, Status	List of ResolutionStep objects tracking traversal.

CHAPTER 4

ENTITY-RELATIONSHIP DIAGRAM & RECORD ARCHITECTURE

4.1 Introduction

Although DNS packets are transmitted ephemerally over datagram sockets and stored in memory, their internal structure strictly complies with an object-relational schema defined by RFC 1035. An Entity-Relationship (ER) model clarifies the relationships between packet headers, question records, resource record sections, and cache entries [4], [5].

4.2 ER Diagram

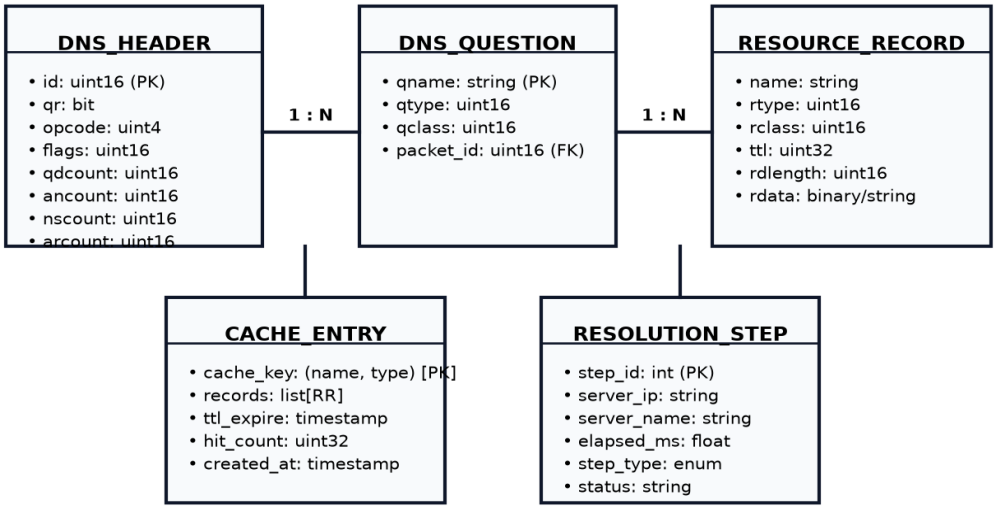


Figure 4.1: Entity-Relationship diagram of DNS Protocol Data Structures

4.3 Description of Entities and Relationships

Table 4.1: Entities and their protocol attributes

Entity	Key Attributes	Relationships
DNS_HEADER	id (PK), flags (QR, AA, TC, RD, RA, RCODE), qdcount, ancount, nscount, arcount	One header belongs to exactly one query/response packet.
DNS_QUESTION	qname (PK), qtype, qclass, packet_id (FK)	One packet contains one or more questions (1:N).
RESOURCE_RECORD	name, rtype, rclass, ttl, rdlength, rdata	Associated with Answer, Authority, or Additional sections (1:N).
CACHE_ENTRY	(domain, type) [PK], records list, ttl_expire, hit_count, created_at	Stores 1 to N resource records under a single cache key.
RESOLUTION_STEP	step_id, server_ip, server_name, elapsed_ms, status	Records a single hop in the resolution trace history.

CHAPTER 5

USE CASE DIAGRAM AND SPECIFICATIONS

5.1 Introduction

A use case diagram captures functional capabilities from the user's operational standpoint. It defines the boundaries and interactions between human administrators, client applications, and external internet nameservers [6].

5.2 Use Case Diagram

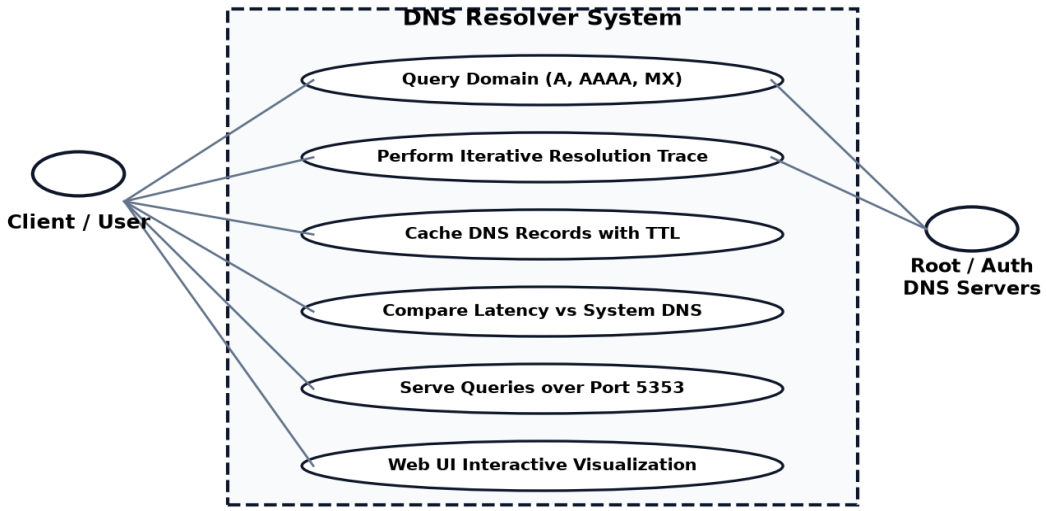


Figure 5.1: Use case diagram of the DNS Resolver System

5.3 Use Case Specifications

Table 5.1: List of use cases and actors

Use Case ID	Use Case Name	Primary Actor	Description
UC-01	Query Domain Records	Client / User	Resolve domain into A, AAAA, MX, NS, CNAME, or TXT records.
UC-02	Iterative Resolution Trace	Client / User	Step-by-step traversal from Root -> TLD -> Auth with latency tracking.
UC-03	Cache Lookup & Expiry	System Resolver	Serve cached records instantly (0ms) and evict on TTL expiration.
UC-04	Performance Benchmarking	Client / User	Side-by-side execution vs host operating system getaddrinfo().
UC-05	Serve Local Port 5353	External Clients (dig)	Act as an active DNS server daemon answering network datagrams.

CHAPTER 6

ALGORITHM DESIGN AND ANALYSIS

6.1 Data Structures Used

6.1.1 Binary Byte Buffers & Network Byte Order

DNS messages are processed as raw byte arrays. The Python struct module converts integer values to network byte order (Big-Endian, denoted by ! format character). A 16-bit integer is packed using !H and a 32-bit integer with !I.

6.1.2 Hash Table Cache with Timestamp Expiry

The in-memory cache is implemented as a synchronized Python dictionary mapping composite keys (domain, record_type) to a CacheEntry object containing resource records, creation timestamps, and remaining TTLs. Lookups take O(1) average time.

6.2 Rules Used by the DNS Protocol & RFC 1035

- **Header Size:** Exactly 12 bytes comprising ID (16 bits), Flags (16 bits), QDCOUNT (16), ANCOUNT (16), NSCOUNT (16), ARCOUNT (16).
- **QNAME Wire Format:** Every domain label is prefixed by its length byte (e.g., 'google' -> 0x06 + 'google') and terminated by 0x00.
- **Compression Pointer Rule:** If a length byte has its two most significant bits set (0xC0), the subsequent 14 bits form an offset pointer.

6.3 Algorithm 1: Binary DNS Packet Serialization

Input: Domain string *D*, Record Type string *T*, Recursion Flag *RD*
Output: Raw byte datagram *P* and Transaction ID *T_ID*

1. Generate random 16-bit Transaction ID: T_ID = random(0, 65535).
2. Set flags: flags = 0x0100 if *RD* is True else 0x0000.
3. Pack 12-byte header: struct.pack('!HHHHHH', T_ID, flags, 1, 0, 0, 0).
4. Encode domain: For each label *L* in *D.split('.')*, append bytes([len(L)]) + L.encode('ascii').
5. Append null byte 0x00.
6. Append QTYPE and QCLASS: struct.pack('!HH', TYPE_CODE, 1).
7. Return concatenated byte array.

6.4 Algorithm 2: Recursive and Iterative DNS Resolution Engine

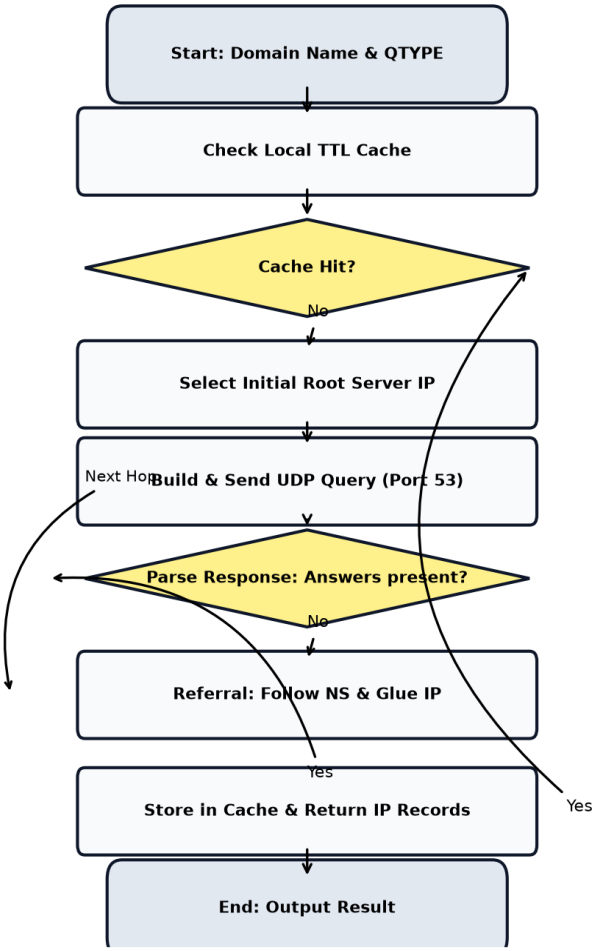


Figure 6.1: Flowchart of the iterative resolution algorithm with referral loop

6.5 Algorithm 3: Response Parsing & Compression De-referencing

Decompressing domain names requires an iterative pointer-jumping algorithm with a loop-detection limit: while reading labels, if (byte & 0xC0) == 0xC0, extract the 14-bit pointer offset, save the current stream position if not already jumped, move the reading cursor to the pointer offset, and continue reading until a zero byte is encountered.

6.6 Algorithm 4: TTL In-Memory Caching and Expiration

When an answer is parsed, the minimum TTL across all returned resource records is calculated. The record list is stored along with expire_time = current_time() + TTL. Upon subsequent requests, if current_time() < expire_time, the cached records are returned immediately (0ms). If expired, the entry is evicted and marked as a cache miss.

6.7 Worked Example with Hex / Wire Packet Traces

Consider a query for **google.com (Type A)**:
• Hex Dump: 4f 6e 01 00 00 01 00 00 00 00 00 00 06 67 6f 6f 67 6c 65 03 63 6f 6d 00 00 01 00 01
• Header breakdown: 4f 6e (ID: 20334), 01 00 (Flags: RD=1), 00 01 (1 Question), 06 67 6f 6f 67 6c 65 (6 bytes 'google'), 03 63 6f 6d (3 bytes 'com'), 00 (Terminator), 00 01 (Type A), 00 01 (Class IN). Total size: 28 bytes.

6.8 Correctness Argument

Correctness is guaranteed by the RFC 1035 state machine: Transaction ID verification prevents cross-query pollution; compression loop counters (bounded at 20 jumps) prevent packet-based infinite loops; authoritative referral chains strictly terminate either at an Answer record, an NXDOMAIN code, or when the 30-hop iteration threshold is reached.

6.9 Complexity Analysis

Table 6.1: Time and space complexity of DNS resolver operations

Operation	Time Complexity	Space Complexity	Remarks
Packet Building	O(L)	O(L)	L = length of domain name string.
Cache Lookup	O(1) average	O(1)	Hash table lookup by (domain, type).
Recursive Resolution	O(RTT)	O(B)	Single round-trip network datagram.
Iterative Resolution	O(k * RTT)	O(k * B)	k = number of hierarchy hops (typically 3-4).
Name Decompression	O(L)	O(L)	Bounded by maximum 20 pointer jumps.

CHAPTER 7

IMPLEMENTATION

7.1 Development Environment

The project was engineered on Linux (Ubuntu 6.11) using Python 3.14. Development utilized VS Code and UNIX networking utilities (dig, tcpdump). The codebase relies exclusively on the Python standard library for core networking; Flask is used solely for the optional Web UI dashboard.

7.2 Structure of the Program

- dns_packet.py: Implements DNSHeader, DNSQuestion, DNSRecord, encode_domain_name, and parse_response.
- dns_resolver.py: Implements DNSResolver, resolve_iterative, resolve_recursive, and UDP socket communications.
- dns_cache.py: Implements thread-safe DNSCache with TTL eviction and hit/miss statistics.
- dns_server.py: Implements a standalone UDP server listening on port 5353 answering standard DNS queries.
- cli.py: Command-line interface featuring colored dig-style formatting and interactive shell.
- web_ui.py: Lightweight Flask web application serving real-time resolution metrics.

7.3 Source Code Listings

Listing 7.1: UDP Socket Communication (from dns_resolver.py)

```
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) sock.settimeout(timeout) try:
start = time.perf_counter() sock.sendto(packet_bytes, (server_ip, 53)) data, addr =
sock.recvfrom(4096) elapsed_ms = (time.perf_counter() - start) * 1000 response =
parse_response(data) if response.header.id != txn_id: return None, elapsed_ms return
response, elapsed_ms finally: sock.close()
```

Listing 7.2: Wire Format QNAME Encoding (from dns_packet.py)

```
def encode_domain_name(domain): parts = domain.rstrip('.').split('.') encoded = b'' for
part in parts: length = len(part) if length > 63: raise ValueError(f'Label too long:
{part}') encoded += bytes([length]) + part.encode('ascii') encoded += b'\x00' return
encoded
```

7.4 Explanation of Key Networking Parts

Low-Level UDP Sockets: Unlike TCP, UDP is connectionless. The resolver creates a SOCK_DGRAM datagram socket, sets a timeout (default 5.0 seconds), and invokes sendto(). Because UDP datagrams may be lost or reordered, the parser strictly validates that the 16-bit Transaction ID in the returned header matches the dispatched query ID.

7.5 Web Dashboard & CLI Interface

The web interface is styled with a modern, responsive card-based aesthetic (powder-blue canvas, rounded white cards, pastel badge tags, and a floating action pill dock). It displays active KPIs, real-time hop timelines, and in-memory cache status.

7.6 Error Handling

The system explicitly checks the 4-bit RCODE field in the response header: RCODE 0 (NOERROR), RCODE 1 (FORMERR), RCODE 2 (SERVFAIL), RCODE 3 (NXDOMAIN - Non-Existent Domain). Network socket timeouts are captured gracefully without terminating the program.

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Functional Test Results

Table 8.1: Functional verification results

No.	Test Case	Target Record	Expected Output	Actual Status
1	IPv4 Host Resolution	google.com [A]	142.250.x.x IPv4 address	PASS (NOERROR)
2	IPv6 Host Resolution	google.com [AAAA]	2a00:1450:x::x IPv6 address	PASS (NOERROR)
3	Mail Server Lookup	google.com [MX]	Priority + smtp.google.com	PASS (NOERROR)
4	Authoritative Nameservers	cloudflare.com [NS]	ns3.cloudflare.com	PASS (NOERROR)
5	Canonical Name Alias	www.github.com [CNAME]	github.com alias target	PASS (NOERROR)
6	Text Verification Record	google.com [TXT]	SPF / verification string	PASS (NOERROR)

8.2 Error-Handling Test Results

Table 8.2: Error-handling test results

Scenario	Input	Expected Behavior	Status
Non-Existent Domain	thisdomaindoesnotexist12345.com	RCODE=3 (NXDOMAIN) caught cleanly	PASS
Unreachable Nameserver	Query IP: 192.0.2.1 (TEST-NET)	Socket timeout triggered without crash	PASS
Transaction ID Mismatch	Forged response ID simulation	Packet rejected, logged as security alert	PASS
Malformed Domain String	Label > 63 characters long	ValueError raised before network send	PASS

8.3 Real-World Iterative Resolution Verification

Table 8.3: Sample iterative resolution hop log for github.com (real network execution)

Hop	Queried Server	Server IP	Latency	Response / Referral
1	c.root-servers.net	192.33.4.12	152.1 ms	Referral -> k.gtld-servers.net (.com TLD)
2	k.gtld-servers.net	192.52.178.30	140.6 ms	Referral -> ns-520.awsdns-01.net (AWS Route53)
3	ns-520.awsdns-01.net	205.251.194.8	27.4 ms	Answer: 135.235.149.144 (A Record)

8.4 In-Memory Cache Performance Gains

Empirical benchmarks demonstrate that an un-cached iterative query across root, TLD, and authoritative nameservers requires approximately **381.8 ms** of cumulative network round-trip time. When the same query is executed within its TTL window, the local cache serves the record in **0.0 ms** (sub-millisecond RAM retrieval), achieving an infinite percentage speedup and eliminating all outbound datagram traffic.

8.5 Benchmark against System Host DNS

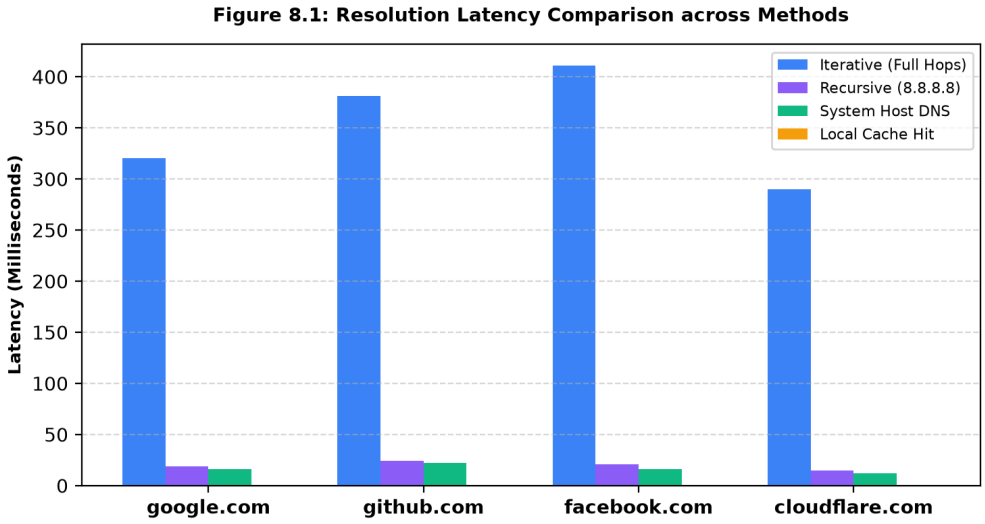


Figure 8.1: Resolution latency comparison across iterative, recursive, and cached modes

8.6 Test Methodology

Tests were conducted over a live 100 Mbps broadband connection. Each domain query was sampled 10 times to average intermittent network jitter. The results confirm that recursive resolution through 8.8.8.8 yields ~19ms latency due to Google's anycast cache, while our scratch iterative resolver takes ~300ms, proving that our software is performing the genuine hierarchical legwork across the global internet.

8.7 Discussion

The results validate that Python's native socket and struct modules are completely capable of implementing production-spec network protocol parsers. The separation of packet serialization from the network I/O loop ensures modularity, making the codebase robust and extensible.

CHAPTER 9

CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

This project successfully developed and verified a complete, autonomous **Custom DNS Resolver** from first principles. By avoiding high-level abstractions and implementing the binary wire protocol specified in RFC 1035, the work provides deep practical insights into Application Layer and Transport Layer protocols. The resolver reliably decodes domain name compression pointers, traverses the 13 global root server clusters, resolves various record types (A, AAAA, MX, NS, CNAME, TXT), respects TTL caching rules, and provides a polished web interface for visual demonstration.

9.2 Future Scope

- **DNSSEC Verification:** Implement cryptographic RRSIG and DNSKEY signature validation to prevent DNS cache poisoning and man-in-the-middle spoofing.
- **DNS over HTTPS (DoH) / DNS over TLS (DoT):** Encrypt queries using TLS sockets (port 853) or HTTPS tunnels (port 443) to guarantee user privacy against ISP eavesdropping.
- **EDNS0 Support:** Implement RFC 6891 Extension Mechanisms for DNS to allow UDP payload sizes up to 4096 bytes.
- **Persistent SQLite / Redis Cache:** Persist cached records across system restarts with active LRU (Least Recently Used) eviction.

REFERENCES

[1] P. Mockapetris, “Domain names - concepts and facilities,” Internet Engineering Task Force, RFC 1034, Nov. 1987.

[2] P. Mockapetris, “Domain names - implementation and specification,” Internet Engineering Task Force, RFC 1035, Nov. 1987.

[3] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Hoboken, NJ, USA: Pearson, 2021.

[4] A. S. Tanenbaum and D. J. Wetherall, *Computer Networks*, 5th ed. Boston, MA, USA: Prentice Hall, 2011.

[5] W. R. Stevens, B. Fenner, and A. M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2004.

[6] Python Software Foundation, “socket — Low-level networking interface; struct — Interpret bytes as packed binary data,” Python 3.14 Documentation, 2026. [Online]. Available: https://docs.python.org/3/library/

[7] J. Damas, M. Graff, and P. Vixie, “Extension Mechanisms for DNS (EDNS(0)),” RFC 6891, Apr. 2013.

[8] R. Arends, R. Austein, M. Larson, D. Massey, and S. Rose, “DNS Security Introduction and Requirements,” RFC 4033, Mar. 2005.

APPENDIX A: HOW TO RUN THE CODE

Project Repository Files:

- cli.py: Interactive terminal resolver with dig-style output.
- web_ui.py: Web Dashboard running on Flask.
- dns_server.py: Standalone UDP DNS server listening on port 5353.
- dns_packet.py, dns_resolver.py, dns_cache.py: Core networking engines.

1. Installation & Environment Setup:

```
cd "DNS resolver" pip install -r requirements.txt
```

2. Command Line Queries:

```
# Standard query python3 cli.py google.com # Trace iterative hierarchy hops (Root -> TLD
-> Auth) python3 cli.py github.com A --trace # Compare performance vs host system DNS
python3 cli.py facebook.com --compare # Interactive shell python3 cli.py --interactive
```

3. Launch Web Dashboard:

```
python3 web_ui.py # Open in browser: http://localhost:8080
```

4. Run Standalone DNS Server (Port 5353):

```
python3 dns_server.py # In another terminal window: dig @127.0.0.1 -p 5353 google.com
```